50

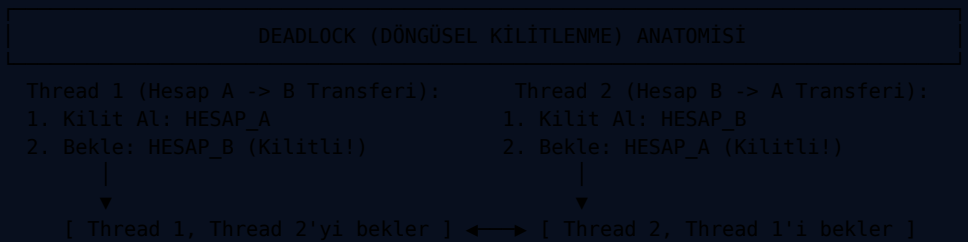
BÖLÜM: EŞZAMANLILIK HATALARI, TEŞHIS & DÜŞÜK SEVIYELI PRIMITIFLER | MODULE: CONCURRENCY BUGS, ANTI-PATTERNS & LOW-LEVEL PRIMITIVES

# Deadlock (Ölümcül Kilitlenme) Forenziği: Coffman Koşulları, Kilit Sıralaması & ThreadMXBean

Deadlock Forensics: Coffman Conditions, Lock Ordering & JVM Detection

## ⚡ Bellek & Eşzamanlılık Akış Şeması

## ⚡ Concurrency & Memory Schematic




## 💡 Aptala Anlatım (ELI5 Metaforu)

## 💡 ELI5 Metaphor

### 🇹🇷 TÜRKÇE (ELI5)

İki inatçı keçinin tek şeritli dar bir tahta köprüde kafa kafaya gelmesini düşünün. Biri 'Önce sen geri adım at', diğeri 'Hayır önce sen çekil' der. İkisi de yolu bırakmaz ve ikisi de köprüyü geçemez. Açlıktan ölene kadar orada dikilirler. Deadlock tam olarak budur: Thread 1, Thread 2'nin elindeki kilidi bekler; Thread 2 ise Thread 1'in elindekini bekler; sistem sonsuza dek donar!

### 🇬🇧 ENGLISH (ELI5)

Two stubborn goats meeting in the middle of a narrow single-lane bridge. Goat A demands Goat B back up; Goat B insists Goat A retreat first. Neither yields, neither moves forward, and both starve to death on the bridge. In Java Deadlock, Thread 1 holds Lock A and waits for Lock B; Thread 2 holds Lock B and waits for Lock A. Neither releases their lock, paralyzing both forever.

## 🔍 MAANG Seviyesi Teknik Derinlik

### 🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Coffman'ın 4 Ölümcül Koşulu ve Çözüm Yolları: Deadlock oluşması için 4 şartın AYNI ANDA gerçekleşmesi gerekir: 1) Mutual Exclusion (Karşılıklı Dışlama): Kaynak sadece tek bir thread tarafından tutulabilir. 2) Hold and Wait (Tut ve Bekle): Thread bir kaynağı tutarken diğerini bekler. 3) No Preemption (Zorla Alma Yok): Kaynak thread'in elinden zorla alınamaz. 4) Circular Wait (Döngüsel Bekleme): Thread'ler kapalı bir döngü zinciri oluşturur (T1->T2->T3->T1). Bu 4 şarttan HERHANGİ BİRİ kırılırsa deadlock imkânsız hale gelir! Endüstride en popüler çözüm: Kilit Sıralaması (Lock Ordering / Global Hierarchy) kuralıdır. Nesne kimliklerine (ör. Account ID veya System.identityHashCode) göre kilitler daima küçükten büyüğe sıralı alınır.

## 🔍 MAANG-Level Technical Deep-Dive

### 🇬🇧 MAANG TECHNICAL DEEP-DIVE

Coffman Conditions & Systematic Remediation: Deadlock requires simultaneous presence of 4 prerequisites: 1) Mutual Exclusion: Resource held exclusively. 2) Hold and Wait: A thread holding a resource requests additional resources. 3) No Preemption: Resources cannot be forcibly expropriated. 4) Circular Wait: A closed chain of threads exists where each waits for a resource held by the next. Breaking ANY ONE condition prevents deadlock. Industry Standard Defense: Global Lock Ordering. By enforcing that locks are ALWAYS acquired in a strictly monotonic hierarchy (e.g. by comparing unique entity IDs or `System.identityHashCode`), circular wait becomes mathematically impossible.

## ⚡ Modern Java 25 LTS Kodu

## ⚡ Modern Java 25 Implementation

```java
// Java 25: Sıralı Kilitleme (Lock Ordering) ile Deadlock'tan Kaçınma
import java.util.concurrent.locks.ReentrantLock;

public class DeadlockFreeTransfer {
    record Account(long id, ReentrantLock lock) {}

    public static void transfer(Account from, Account to, double amount) {
        // Kilit sıralaması: Her zaman ID'si KÜÇÜK olanı önce kilitle!
        Account firstLock = from.id() < to.id() ? from : to;
        Account secondLock = from.id() < to.id() ? to : from;

        firstLock.lock().lock();
        try {
            secondLock.lock().lock();
            try {
                System.out.printf("Transfer yapıldı: %d -> %d tutar: %.2f%n",
                    from.id(), to.id(), amount);
            } finally {
                secondLock.lock().unlock();
            }
        } finally {
            firstLock.lock().unlock();
        }
    }

    void main() {
        var accA = new Account(1001, new ReentrantLock());
        var accB = new Account(2002, new ReentrantLock());

        // Ters yönlü transferler aynı anda gelse bile ASLA deadlock olmaz!
        transfer(accA, accB, 50.0);
        transfer(accB, accA, 30.0);
        System.out.println("Transferler güvenle tamamlandı.");
    }
}
```

### ⚠️ Deadlock, Race Condition & Mülakat Tuzağı

**🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE**

tryLock(timeout) kullanarak deadlock riskini azaltabilirsiniz; ancak kilit alınamadığında daha önce alınan kilitleri bırakmayı (unlock) unutursanız livelock yaratırsınız. Kilit sıralaması (Lock Ordering) matematiksel olarak en sağlam yoldur.

### ⚠️ Concurrency Hazard & Interview Trap

**🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD**

Mitigating deadlocks via `tryLock(timeout)` is fragile: if a thread fails to acquire the second lock but retains the first, it may induce Livelock. Monotonic lock ordering remains the definitive architectural defense.

### 🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Deadlock döngüsel kilit beklemesinden doğar. Kilitleri daima sabit bir sıralamayla (küçük ID'den büyük ID'ye) alarak döngüyü kırın.

### 🎯 Key Takeaways for the Interview

- Deadlock stems from circular waiting. Enforce strict lock ordering across all execution paths to break the cycle mathematically.